4-byte Integer Hashing

The hashes on this page (with the possible exception of HashMap.java's) are all public domain. So are the ones on Thomas Wang's page. Thomas recommends citing the author and page when using them.

Thomas Wang has an integer hash using multiplication that's faster than any of mine on my Core 2 duo using gcc -O3, and it passes my favorite sanity tests well. I've had reports it doesn't do well with integer sequences with a multiple of 34.

```
uint32_t hash( uint32_t a)
    a = (a ^ 61) ^ (a >> 16);
    a = a + (a << 3);
    a = a ^ (a >> 4);
    a = a * 0x27d4eb2d;
    a = a ^ (a >> 15);
    return a;
}
```

Thomas has 64-bit integer hashes too. I don't have any of those yet.

Here's a way to do it in 6 shifts:

```
uint32_t hash( uint32_t a)
{
    a = (a+0x7ed55d16) + (a<<12);
    a = (a^0xc761c23c) ^ (a>>19);
    a = (a+0x165667b1) + (a<<5);
    a = (a+0xd3a2646c) ^ (a<<9);
    a = (a+0xfd7046c5) + (a<<3);
    a = (a^0xb55a4f09) ^ (a>>16);
    return a;
}
```

Or 7 shifts, if you don't like adding those big magic constants:

```
uint32_t hash( uint32_t a)
{
    a -= (a<<6);
    a ^= (a>>17);
    a -= (a<<9);
    a ^= (a<<4);
    a -= (a<<3);
    a ^= (a<<10);
    a ^= (a>>15);
    return a;
}
```

Thomas Wang has a function that does it in 6 shifts (provided you use the low bits, hash & (SIZE-1), rather than the high bits if you can't use the whole value):

```
uint32_t hashint( uint32_t a)
{
    a += ~(a<<15);
    a ^=  (a>>10);
    a +=  (a<<3);
    a ^=  (a>>6);
    a += ~(a<<11);
    a ^=  (a>>16);
}
```

Here's a 5-shift one where you have to use the high bits, hash >> (32-logSize), because the low bits are hardly mixed at all:

```
uint32_t hashint( uint32_t a)
{
    a = (a+0x479ab41d) + (a<<8);
    a = (a^0xe4aa10ce) ^ (a>>5);
    a = (a+0x9942f0a6) - (a<<14);
    a = (a^0x5aedd67d) ^ (a>>3);
    a = (a+0x17bea992) + (a<<7);
    return a;
}
```

Here's one that takes 4 shifts. You need to use the bottom bits, and you need to use at least the bottom 11 bits. It doesn't achieve avalanche at the high or the low end. It does pass my integer sequences tests, and all settings of any set of 4 bits usually maps to 16 distinct values in bottom 11 bits.

```
uint32_t hashint( uint32_t a)
{
    a = (a^0xdeadbeef) + (a<<4);
    a = a ^ (a>>10);
    a = a + (a<<7);
    a = a ^ (a>>13);
    return a;
}
```

And this one isn't too bad, provided you promise to use at least the 17 lowest bits. Passes the integer sequence and 4-bit tests.

```
uint32_t hashint( uint32_t a)
{
    a = a ^ (a>>4);
    a = (a^0xdeadbeef) + (a<<5);
```

```
    a = a ^ (a>>11);
    return a;
}
```

More Wordy Stuff

Adam Zell points out that this hash is used by the HashMap.java:

```
private static int newHash(int h) {
    // This function ensures that hashCodes that differ only by
    // constant multiples at each bit position have a bounded
    // number of collisions (approximately 8 at default load factor).
    h ^= (h >>> 20) ^ (h >>> 12);
    return h ^ (h >>> 7) ^ (h >>> 4);
}
```

Although this hash leaves something to be desired (h & 2047 uses only 1/8 of the buckets for sequences incremented by 8), it does bring up an interesting point: full avalanche is stronger than what you really need for a hash function. All you really need is that the entropy in your keys be represented in the bits of the hash value that you use. Often you can show this by matching every differing input bit to a distinct bit that it changes in the portion of the hash value that you use.

One very non-avalanchy example of this is CRC hashing: every input bit affects only some output bits, the ones it affects it changes 100% of the time, and every input bit affects a different set of output bits. If the input bits that differ can be matched to distinct bits that you use in the hash value, you're golden. Otherwise you're not.

4-byte integer hash, half avalanche

Full avalanche says that differences in any input bit can cause differences in any output bit. A weaker property is also good enough for integer hashes if you always use the high bits of a hash value: **every input bit affects its own position and every higher position**. I'll call this **half avalanche**. (Multiplication is like this, in that every bit affects only itself and higher bits. But multiplication can't cause every bit to affect EVERY higher bit, especially if you measure "affect" by both - and ^.) Half-avalanche is sufficient: if you use the high n bits and hash 2^n keys that cover all possible values of n input bits, all those bit positions will affect all n high bits, so you can reach up to 2^n distinct hash values. It's also sometimes necessary: if you use the high n+1 bits, and the high n input bits only affect their position and greater, and you take the 2^{n+1} keys differing in the high n bits plus one other bit, then the only way to get over 2^n hash values is if that one other input bit affects position n+1 from the top. For all n less than itself. So it has to affect itself and all higher bits.

Actually, that wasn't quite right. Half-avalanche says that an input bit will change its output bit (and all higher output bits) half the time. But if the later output bits are all dedicates to representing other input bits, you want this output bit to be affected 100% of the time by this input bit, not 50% of the time. This doesn't entirely kill the idea though. If every bit affects itself and all higher bits, plus a couple lower bits, and you use just the high-order bits, then the lowest high-order bit you use still contains entropy from several differing input bits. So it might work. Hum. Better check how this does in practice!

Similarly for low-order bits, it would be enough for every input bit to affect only its own position and all lower bits in the output (plus the next few higher ones). Half-avalanche is easier to achieve for high-order bits than low-order bits because a*=k (for odd k), a+=(a<<k), a-=(a<<k), a^=(a<<k) are all permutations that affect higher bits, but only a^=(a>>k) is a permutation that affects lower bits. (There's also table lookup, but unless you get a lot of parallelism that's going to be slower than shifts.)

Here's a 5-shift function that does half-avalanche in the high bits:

```
uint32_t half_avalanche( uint32_t a)
{
    a = (a+0x479ab41d) + (a<<8);
    a = (a^0xe4aa10ce) ^ (a>>5);
    a = (a+0x9942f0a6) - (a<<14);
    a = (a^0x5aedd67d) ^ (a>>3);
    a = (a+0x17bea992) + (a<<7);
    return a;
}
```

Every input bit affects itself and all higher output bits, plus a few lower output bits. I hashed sequences of n consecutive integers into an n-bucket hash table, for n being the powers of 2 2^1 .. 2^{20}, starting at 0, incremented by odd numbers 1..15, and it did OK for all of them. Also, for "differ" defined by +, -, ^, or ^~, for nearly-zero or random bases, inputs that differ in any bit or pair of input bits will change each equal or higher output bit position between 1/4 and 3/4 of the time. Here's a table of how the ith input bit (rows) affects the jth output bit (columns) in that hash (single bit differences, differ defined as ^, with a random base):

51	46	48	51	55	52	45	51	53	50	50	50	50	50	49	50	50	51	50	50	50	49	50	50	51	50	56	50	44	65	50	44
51	32	46	52	54	55	55	51	45	51	50	53	51	50	50	46	50	50	53	51	50	50	48	50	50	52	50	56	50	44	65	50
52	47	38	50	50	55	48	49	51	47	50	50	51	51	50	50	47	50	50	53	50	50	50	48	50	50	52	50	56	50	44	65
85	60	43	33	51	48	57	51	51	50	45	51	53	53	50	50	50	45	50	51	55	50	50	50	47	50	50	53	50	56	50	44
15	93	58	45	32	50	48	58	50	51	50	50	49	50	50	50	50	50	50	50	51	50	50	50	50	47	50	50	53	50	56	50
54	61	62	53	51	39	49	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	49	50	50	51	50	56
51	68	40	52	54	40	38	50	51	51	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	48	50	50	52	50
51	32	81	53	55	42	48	50	50	53	50	50	49	50	50	49	50	50	51	50	50	50	50	50	50	50	50	50	47	50	50	50
100	50	44	64	55	54	45	54	50	50	53	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	46	50	50

0	100	50	47	60	55	51	49	63	51	49	50	50	50	50	50	50	49	50	50	50	50	50	50	50	50	50	50	50	50	42	50
0	0	100	50	48	64	50	49	51	61	50	50	52	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	47	42
0	0	0	100	51	43	63	49	58	50	60	50	50	53	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	48
0	0	0	0	100	50	48	60	50	54	49	49	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
0	0	0	0	0	100	50	45	70	50	54	51	56	50	50	52	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
0	0	0	0	0	0	100	50	37	74	50	59	50	56	50	50	52	50	50	50	50	50	50	50	50	50	50	50	50	51	50	50
0	0	0	0	0	0	0	100	50	38	72	50	60	50	56	50	50	52	50	50	50	50	50	50	50	50	50	50	50	50	50	50
0	0	0	0	0	0	0	0	100	50	40	67	50	61	50	56	50	50	52	50	50	50	50	50	50	50	50	50	50	50	50	50
0	0	0	0	0	0	0	0	0	100	50	50	62	50	51	50	55	50	50	51	50	50	50	50	50	50	50	50	50	50	50	51
0	0	0	0	0	0	0	0	0	0	100	50	43	67	50	57	50	56	50	50	52	50	50	50	50	50	50	50	51	50	53	50
0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	56	50	56	50	50	52	50	50	50	50	50	50	50	50	50	52
0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	52	50	50	50	50	50	50	50	50	50
0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	51	50	50	50	50	50	50	50	50
0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	52	50	50	50	50	45	50	50
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	53	50	50	50	50	40	50
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	54	50	55	50	44	66
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	54	50	56	50	44
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	56	50	50	54	50	55	50
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	60	50	50	55	50	55
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	52	50	51	53	50
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	66	50	57	50	61	50	50	51
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	44	70	50	57	50	62	50	50
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	100	50	38	75	50	62	50	73	50

If you use high-order bits for hash values, adding a bit to the hash value to double the size of the hash table will add a low-order bit, so old bucket 0 maps to the new 0,1, old bucket 1 maps to the new 2,3, and so forth. They overlap. It's not as nice as the low-order bits, where the new buckets are all beyond the end of the old table. Also, using the n high-order bits is done by (a>>(32-n)), instead of (a&((1<<n)-1)), note that >> takes 2 cycles while & takes only 1. But, on the plus side, if you use high-order bits for buckets and order keys inside a bucket by the full hash value, and you split the bucket, all the keys in the low bucket precede all the keys in the high bucket (Shalev '03, split-ordered lists). Incrementally splitting the table is still feasible if you split high buckets before low buckets; that way old buckets will be empty by the time new buckets take their place.

4-byte integer hash, full avalanche

I was able to do it in 6 shifts.

```
uint32_t hash( uint32_t a)
{
    a = (a+0x7ed55d16) + (a<<12);
    a = (a^0xc761c23c) ^ (a>>19);
    a = (a+0x165667b1) + (a<<5);
    a = (a+0xd3a2646c) ^ (a<<9);
    a = (a+0xfd7046c5) + (a<<3);
    a = (a^0xb55a4f09) ^ (a>>16);
    return a;
}
```

These magic constants also worked: 0x7fb9b1ee, 0xab35dd63, 0x41ed960d, 0xc7d0125e, 0x071f9f8f, 0x55ab55b9 .

For one or two bit diffs, for "diff" defined as subtraction or xor, for random or nearly-zero bases, every output bit changes with probability between 1/4 and 3/4. I also hashed integer sequences incremented by odd 1..31 times powers of two; low bits did marvelously, high bits did sorta OK. Here's the table for one-bit diffs on random bases with "diff" defined as XOR:

50	47	50	51	50	50	50	49	42	50	50	50	49	50	50	50	50	47	50	49	58	50	51	46	62	50	55	50	45	63	51	45
50	50	50	50	50	50	50	50	50	48	50	50	50	50	50	50	50	50	50	50	50	53	50	51	47	59	50	52	50	48	63	51
54	50	49	50	50	51	49	50	50	49	45	50	50	50	47	50	54	50	51	46	50	48	58	49	50	43	64	51	59	49	43	70
50	52	50	50	50	50	51	49	50	50	50	47	50	50	50	49	50	52	50	51	48	50	49	54	50	51	46	63	51	58	51	42
42	50	50	50	50	50	50	50	49	50	50	50	47	50	50	50	41	50	50	50	50	50	50	50	55	50	50	48	60	50	52	50
48	43	50	51	50	50	50	50	51	49	50	50	50	46	50	50	46	44	50	51	50	50	46	50	48	56	51	52	44	67	50	60
49	48	43	50	51	50	50	50	50	51	49	50	50	50	47	50	49	49	42	50	52	50	50	45	50	48	60	49	51	44	65	51
50	50	50	53	50	49	50	50	50	50	50	50	50	50	50	50	50	50	50	47	50	54	50	51	45	50	47	59	49	51	46	64
50	50	50	50	51	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	49	50	50	50	50	49	50	50	53	50	50	47
52	50	50	50	50	52	50	50	50	50	50	50	50	50	50	50	52	50	50	50	50	48	50	52	50	50	46	50	49	55	50	50
50	51	50	50	50	50	52	50	49	50	50	50	50	50	50	50	50	51	50	50	50	50	48	50	50	50	50	49	50	50	53	50
47	50	50	50	50	50	50	52	50	49	50	50	50	50	50	50	47	50	50	50	50	50	50	48	50	51	50	50	49	50	50	56

50	47	50	51	50	50	50	50	52	50	49	50	50	50	50	50	50	47	50	51	50	50	50	50	48	50	53	50	50	47	50	49
51	50	49	50	50	50	50	50	50	52	50	50	50	50	50	50	51	50	49	50	50	50	50	50	50	48	51	51	50	50	47	50
50	51	51	46	50	52	50	51	50	50	52	50	49	50	50	50	50	51	51	46	50	52	50	49	50	50	48	51	50	51	51	49
50	50	50	51	46	50	52	50	50	50	50	51	50	50	50	50	50	50	50	51	46	50	52	50	50	49	50	48	50	53	49	51
54	50	50	50	50	49	50	50	50	50	50	50	51	50	50	50	55	50	50	50	50	49	50	50	50	50	50	50	48	50	50	50
50	54	50	50	50	50	46	50	52	50	50	50	50	51	50	50	50	54	50	50	50	50	45	50	53	50	50	50	50	47	50	50
52	50	53	50	50	50	50	46	50	51	50	50	50	50	51	50	53	50	53	50	50	50	50	45	50	52	50	50	49	50	48	50
50	48	49	49	50	50	50	50	54	50	50	50	50	50	50	50	50	45	50	51	50	50	49	51	43	50	54	50	49	49	50	45
50	55	43	50	46	50	52	47	50	47	50	50	53	50	50	51	50	45	65	51	55	50	48	63	50	52	50	47	63	50	50	46
57	50	56	51	50	46	50	52	48	50	47	50	50	54	50	50	57	50	44	66	51	56	50	42	64	50	59	50	42	65	49	50
51	60	50	56	51	50	46	50	52	47	50	48	50	50	53	50	51	60	50	44	66	51	55	50	47	62	50	52	50	47	62	50
60	50	51	50	55	50	51	46	50	52	50	50	48	50	50	51	60	50	51	50	45	64	51	55	50	48	63	50	52	50	48	63
45	63	51	55	50	56	51	50	45	50	52	50	50	47	50	50	45	63	51	55	50	44	65	51	55	50	42	69	51	59	49	42
49	45	63	50	56	50	55	50	51	46	50	52	50	50	47	50	49	45	63	50	56	50	45	65	51	55	50	42	63	51	59	51
49	49	45	63	50	56	50	56	50	51	47	50	52	50	50	48	49	49	45	63	50	56	50	44	65	51	55	50	48	63	50	52
65	49	50	45	64	50	56	50	56	50	50	47	50	51	50	50	65	49	50	45	64	50	56	50	44	65	51	55	50	42	69	51
45	65	49	50	45	64	51	56	50	55	50	50	47	50	51	49	45	65	49	50	45	64	51	56	50	45	65	51	55	50	42	64
50	44	66	49	50	45	63	51	56	50	56	50	50	47	50	51	50	44	66	49	50	45	63	51	56	50	44	65	51	55	50	48
55	49	44	68	50	50	45	65	50	55	50	56	50	50	47	50	55	49	44	68	50	50	45	65	50	55	50	44	67	51	55	50
51	60	50	39	73	49	52	43	70	51	60	50	60	51	50	45	51	60	50	39	73	49	52	43	70	51	60	50	40	72	51	60

If you don't like big magic constants, here's another hash with 7 shifts:

```
uint32_t hashint( uint32_t a)
{
    a -= (a<<6);
    a ^= (a>>17);
    a -= (a<<9);
    a ^= (a<<4);
    a -= (a<<3);
    a ^= (a<<10);
    a ^= (a>>15);
    return a;
}
```

I've confirmed this does well with sequences incremented by common amounts whether you use the high or low bits of the hash. And it does avalanche if one or two input bits differ, for a variety of base input values, with "differ" defined as + ^ - or ~^. For one-bit input differences on top of a random base value with "differ" defined as ^, each input bit changes each output bit with probability between .39 and .73 (for random bases with diff defined as XOR). Specifically, this is the percentage of the times the ith input bit (rows) changed the jth output bit (columns):

50	50	50	50	50	50	50	50	50	51	50	50	50	49	50	50	51	50	50	50	50	50	45	50	51	51	49	50	55	51	52	45
50	50	50	50	50	50	50	50	49	50	51	50	50	50	49	50	50	50	50	50	50	50	50	46	50	51	50	49	50	55	51	52
49	50	50	50	50	49	50	50	50	49	50	51	50	50	50	50	50	50	50	50	51	51	50	50	45	50	51	51	49	50	55	51
55	49	50	50	49	50	50	50	50	50	49	50	51	50	51	50	50	50	50	51	50	51	50	50	50	47	50	50	49	48	50	55
51	54	49	50	50	50	50	50	50	50	50	50	50	51	50	50	50	49	50	50	51	50	50	50	50	50	47	50	50	50	48	50
50	51	54	49	50	50	50	50	50	50	50	50	50	50	51	50	50	54	49	50	50	51	50	51	50	50	50	47	50	50	50	48
44	50	50	51	50	50	50	50	50	50	50	50	50	50	50	50	50	50	51	50	50	50	51	50	50	50	50	50	48	50	50	50
49	44	50	50	51	50	50	50	50	50	50	50	50	50	50	50	50	50	50	51	50	50	50	50	50	50	50	50	50	48	50	50
50	49	44	50	50	52	50	50	50	50	50	50	50	50	50	50	50	44	50	50	52	50	51	50	51	50	50	50	50	50	48	50
54	50	50	47	50	50	52	49	50	50	50	50	50	50	50	50	50	50	47	50	50	52	50	50	50	51	50	50	50	50	50	48
51	52	50	50	48	50	50	51	50	50	50	50	50	50	50	50	50	50	50	49	50	50	51	50	50	50	50	50	50	51	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	51	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	51	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	51	50	51	50	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	51	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
52	49	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	52	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50	50
50	50	52	49	50	50	50	50	50	50	50	50	50	50	50	50	50	52	49	50	50	50	50	50	50	50	50	50	50	50	50	50

44	50	50	51	49	50	50	49	50	50	50	50	50	50	50	50	50	50	51	49	50	50	51	50	50	50	50	50	50	50	50	50
49	43	50	50	51	49	50	50	50	50	50	50	50	50	50	50	50	50	50	51	49	50	50	50	50	50	50	50	50	50	50	50
50	49	41	50	50	52	50	50	50	50	50	50	50	50	50	50	50	41	50	50	52	50	50	50	51	50	50	50	50	50	50	50
44	66	48	58	48	50	53	49	50	50	50	50	50	50	49	50	51	48	58	48	50	53	49	50	50	53	49	50	52	49	50	49
52	45	65	48	59	48	50	53	49	50	49	49	50	50	50	49	50	65	48	59	48	50	53	49	50	51	53	49	50	51	48	50
50	52	45	66	48	59	48	50	53	50	50	50	49	50	50	50	49	45	66	48	59	48	50	53	50	50	50	53	49	50	52	49
65	50	52	44	66	48	59	47	50	53	49	50	49	50	50	50	50	52	44	66	48	59	47	50	53	49	50	51	53	49	50	51
45	68	50	52	43	68	48	59	48	51	53	50	50	49	50	50	50	50	52	43	68	48	59	48	51	53	50	50	51	53	49	50
49	39	73	51	53	41	73	47	65	45	50	55	49	50	49	49	50	73	51	53	41	73	47	65	45	50	55	49	50	51	54	48

The following operations and shifts cause inputs that differ in 1 or 2 bits to differ with probability between 1/4 and 3/4 in each output bit. (k=1..31 is += <<k, k=33..63 is -= <<(k-32), 65..95 is ^= <<(k-64), and 97..127 is ^= >>(k-96).) I put a * by the line that represents the hash above.

```
38 113  41  68  35  74 111 *
38 113  42  69  35  73 112
38 114   9 100  35 107  46
38 114  11  66   8  68 112
38 114  42  69  35  73 112
38 114  78  37  71  35 111
39 113  41  68   2  74 112
39 114   9 101   2 107  17
39 114   9 101   2 107  49
39 114  37  99  39 109  50
39 115  36  67  38  44 112
39 115  37  70  35 110  11
39 115  41  74  36  67 111
39 116   4 104   6 107  16
39 116  10 101   8  75 113
40 113  12  99  39  69 112
40 113  13  99   6  69 113
40 113  38 101   2 106  16
40 113  38 101   2 106  48
40 114   3 102   8 109  15
40 114  37  99   7  77 113
41 113  11 100   7  69 111
42 114  44  99  38  72 113
43 115   7 101   3 109  48
44 114  36 105  38 108  16
44 114  37 102  35 107  16
44 114  41 101   2 109  16
45 113  37 102   3 108  47
45 113  37 105  35 104  17
45 113  37 105  35 104  47
45 113  39  99  37  76 111
45 113  42 101   2 109  46
45 113  42 101   2 109  50
46 113  42 101  35 110  47
46 113  42 101  35 110  50
```

Another hash is [Thomas Wang's function](),

```
uint32_t hashint( uint32_t a)
{
    a += ~(a<<15);
    a ^=  (a>>10);
    a +=  (a<<3);
    a ^=  (a>>6);
    a += ~(a<<11);
    a ^=  (a>>16);
}
```

I got the idea of adding a constant from looking at what affect his ~ had. His function passed only half of my tests for full avalanche. If you use it, the low bits are mixed better than the high bits. The low bits do quite well on sequences incremented by a constant amount. For random bases and diff defined as XOR, it did kinda OK, every input bit affects every output bit with probability between .36 and .76, specifically:

50	49	50	50	50	50	50	54	49	49	51	50	50	50	50	50	50	50	53	50	50	50	45	63	51	45	65	50	43	67	50	45
50	50	50	48	50	50	50	50	55	49	50	50	50	50	50	50	50	50	50	54	50	50	50	45	63	51	45	65	50	43	67	50
50	50	50	50	47	50	50	50	49	56	49	50	50	50	50	50	49	50	50	50	55	49	50	50	45	63	50	45	65	50	43	67
50	50	50	50	50	48	50	50	50	49	56	50	50	50	50	50	50	49	49	50	50	55	50	50	50	45	63	51	45	65	50	43
50	50	50	50	50	50	48	50	50	50	49	53	50	50	50	50	50	50	50	49	50	50	55	50	50	50	45	63	51	45	65	50
50	50	50	50	50	50	50	48	50	50	50	50	53	50	50	50	50	50	50	50	49	50	50	55	49	50	50	45	64	50	45	64
50	50	50	50	50	50	50	50	47	50	50	50	50	51	50	50	50	50	50	50	50	49	50	50	54	50	50	50	45	65	50	46
50	50	50	50	51	50	50	50	50	47	50	50	50	50	51	51	50	50	50	50	49	50	49	50	50	55	49	50	50	43	67	52
39	50	50	50	50	50	50	50	50	50	47	50	50	50	50	52	39	50	50	50	50	50	50	49	50	50	55	49	50	50	43	68
47	39	50	50	50	50	50	50	50	50	50	47	50	50	50	50	47	39	50	50	50	50	50	50	49	50	50	55	47	50	50	42
50	52	51	50	50	50	50	51	50	50	50	50	50	50	50	50	50	48	40	50	50	50	50	49	50	49	50	50	57	47	50	50
49	50	50	46	50	50	50	50	50	50	51	50	50	50	50	50	50	50	48	40	50	50	50	50	50	50	49	50	50	57	47	50
50	49	50	49	43	50	50	50	50	50	50	50	50	50	50	50	49	50	50	48	41	50	50	50	50	50	50	49	48	50	58	47

51	50	50	50	50	46	50	50	50	50	51	50	50	50	50	50	50	50	50	50	48	41	50	50	50	50	52	50	49	48	50	58
51	50	50	49	50	50	45	50	50	50	51	47	50	50	50	50	50	50	49	50	50	48	40	50	50	50	50	57	50	49	51	50
50	50	50	50	49	50	50	47	50	50	50	50	47	50	50	50	50	50	50	50	50	50	49	43	50	50	50	50	55	50	50	49
50	50	50	50	50	49	50	50	46	50	50	50	50	48	50	50	50	50	50	50	50	50	50	48	41	50	50	50	50	56	48	50
51	47	50	50	46	50	50	50	50	56	49	49	50	50	52	50	51	53	50	50	54	50	50	50	45	63	50	45	65	50	43	67
49	51	47	50	50	46	50	50	50	49	57	49	49	50	50	52	49	51	53	50	50	54	50	50	50	45	63	51	45	65	50	43
54	49	50	47	50	50	45	50	51	50	49	56	49	49	50	50	54	49	50	53	50	50	54	50	50	50	45	63	51	45	65	50
50	54	49	50	47	50	50	46	50	50	50	49	56	49	50	50	50	54	49	50	53	50	50	54	49	50	50	45	64	50	45	64
50	50	54	49	51	47	50	50	45	50	50	50	50	53	50	50	50	50	54	49	51	53	50	50	55	50	49	50	44	65	50	46
50	50	50	54	49	50	47	50	50	45	50	50	51	50	53	49	50	50	50	54	49	50	53	50	50	55	49	50	50	43	67	52
67	50	50	50	55	49	50	47	50	50	45	50	50	50	50	53	67	50	50	50	55	49	50	53	50	50	54	49	50	50	43	68
43	67	50	50	50	55	49	50	47	50	50	45	50	50	50	50	43	67	50	50	50	55	49	50	53	50	50	55	48	50	50	42
50	43	67	50	50	50	54	49	50	47	50	50	43	50	50	50	50	43	67	50	50	50	54	49	50	53	50	50	56	47	50	50
65	50	43	67	50	50	50	55	49	50	47	50	50	42	50	50	65	50	43	67	50	50	50	55	49	50	53	50	50	57	47	50
44	65	50	43	67	50	50	50	54	49	50	46	50	50	44	50	44	65	50	43	67	50	50	50	54	49	50	54	50	50	57	47
50	45	65	50	43	67	50	50	50	55	51	50	45	50	50	44	50	45	65	50	43	67	50	50	50	55	51	50	55	50	50	57
68	50	44	64	50	43	67	51	50	50	46	65	50	44	50	50	68	50	44	64	50	43	67	51	50	50	46	65	50	56	50	50
43	72	50	47	69	50	43	71	50	50	51	44	70	50	47	50	43	72	50	47	69	50	43	71	50	50	51	44	70	50	53	50
50	36	76	53	39	74	50	37	75	47	50	50	37	76	53	39	50	36	76	53	39	74	50	37	75	47	50	50	37	76	53	61